جزوه درجه یک سیستم عامل

فصل ۴ - بخش ۵: نخ‌سازی ضمنی (Implicit Threading)

در بخش‌های قبلی یاد گرفتیم که چطور به صورت "صریح" (Explicit) یک نخ بسازیم. این کار سخت، پر از خطا و پیچیده است، مخصوصاً وقتی تعداد نخ‌ها زیاد شود.

نخ‌سازی ضمنی یک رویکرد مدرن است که می‌گوید: "برنامه‌نویس نباید درگیر مدیریت نخ شود."

ایده اصلی:

- **برنامه‌نویس (شما):** فقط "وظایف" (Tasks) یا بخش‌هایی از کد که می‌توانند موازی اجرا شوند را شناسایی و تعریف می‌کنید. (مثل لیست کردن کارهایی که باید انجام شود).
- **پیمانکار (کامپایلر یا کتابخانه):** مسئولیت کامل ساختن، مدیریت، زمان‌بندی و تخصیص وظایف به نخ‌ها را بر عهده می‌گیرد. (او خودش کارگرها را استخدام و مدیریت می‌کند).

این "پیمانکارها" معمولاً از مدل "چند-به-چند" (که در بخش ۳ دیدیم) استفاده می‌کنند تا هزاران وظیفه (Tasks) را روی تعداد کمی نخ (Threads) مدیریت کنند.

بیایید با مهم‌ترین و رایج‌ترین این "پیمانکارها" آشنا شویم:

۴-۵-۱: انبارهای نخ (Thread Pools)

این ساده‌ترین و پرکاربردترین مدل نخ‌سازی ضمنی است.

ایده: به جای اینکه برای هر کار جدید یک کارگر جدید استخدام (یک نخ `create`) و در پایان کار اخراجش کنیم (نخ `destroy`)، یک "انبار" یا "تیم" از کارگرهای آماده به کار می‌سازیم.

1. در ابتدای برنامه، تعدادی نخ (مثلاً ۱۰ نخ) ایجاد می‌کنیم و آن‌ها را در "انبار" (Pool) قرار می‌دهیم. این نخ‌ها بی‌کار می‌مانند و **منتظر کار** می‌نشینند.
2. وقتی یک "وظیفه" (Task) جدید می‌رسد (مثلاً یک درخواست به سرور وب)، سیستم آن را به "صف" (Queue) وظایف اضافه می‌کند.
3. یک نخ بی‌کار از انبار، وظیفه را از صف برمی‌دارد و آن را اجرا می‌کند.
4. **نکته کلیدی:** بعد از اتمام کار، نخ "اخراج" نمی‌شود، بلکه به انبار برمی‌گردد و دوباره منتظر کار بعدی می‌ماند.

منافع (سوال امتحانی):

1. **سرعت:** سرویس‌دهی بسیار سریع‌تر است، چون "هزینه ساخت نخ" را نداریم. نخ از قبل آماده است.
2. **کنترل منابع:** تعداد کل نخ‌های سیستم "محدود" است (به اندازه انبار). این کار جلوی مصرف بی‌رویه منابع سیستم را می‌گیرد.
3. **تفکیک وظیفه از اجرا:** وظیفه (Task) از نحوه اجرای آن جدا می‌شود. ما می‌توانیم برای اجرای وظایف استراتژی‌های مختلفی بچینیم (مثلاً وظیفه با تأخیر اجرا شود).

پیاده‌سازی‌ها:

- **ویندوز:** با تابع `QueueUserWorkItem` یک وظیفه را به انبار نخ پیش‌فرض ویندوز می‌سپارید.
- **جاوا:** این مدل در جاوا فوق‌العاده است. از طریق `Executors` می‌توانید انبارهای نخ مختلف بسازید:
  - `newFixedThreadPool(10)`: یک انبار با دقیقاً ۱۰ نخ ثابت می‌سازد.
  - `newCachedThreadPool()`: یک انبار هوشمند که بسته به نیاز، نخ به آن اضافه یا از آن کم می‌کند.
  - `newSingleThreadExecutor()`: یک انبار با فقط یک نخ (برای اجرای ترتیبی کارها).

۴-۵-۲: مدل Fork-Join

این مدل یک نسخه "همگام" (Synchronous) از نخ‌سازی ضمنی است که برای الگوریتم‌های **"تقسیم و غلبه" (Divide and Conquer)** عالی است.

ایده (مثل الگوریتم QuickSort یا MergeSort):

1. **Fork (انشعاب):** نخ اصلی (والد) یک وظیفه بزرگ را به دو زیر-وظیفه کوچکتر تقسیم می‌کند و آن‌ها را به صورت موازی "Fork" می‌کند (شروع به اجرا می‌کند).
2. این زیر-وظیفه‌ها هم به نوبه خود می‌توانند Fork کنند و کوچکتر شوند (یک ساختار درختی).
3. **Join (الحاق):** نخ والد **منتظر** می‌ماند تا هر دو زیر-وظیفه کارشان تمام شود.
4. وقتی نتایج برگشتند، والد آن‌ها را با هم "Join" (ترکیب) می‌کند.

پیاده‌سازی در جاوا (Fork-Join Framework):

- **`ForkJoinPool`:** یک انبار نخ مخصوص این کار است.
- **`RecursiveTask`:** کلاسی که شما باید ارث‌بری کنید و منطق "تقسیم" (Fork) و "حل مستقیم" (Compute) را در آن بنویسید.
- **دزدی کار (Work-Stealing):** این یک تکنیک هوشمندانه برای تعادل بار (Load Balancing) است. هر نخ یک صف وظایف برای خودش دارد. اگر یک نخ بی‌کار شد (صفش خالی شد)، به جای منتظر ماندن، می‌رود و از انتهای صف یک نخ دیگر که پرکار است، کار "می‌دزدد" و آن را انجام می‌دهد. این کار باعث می‌شود تمام هسته‌های CPU مشغول بمانند.

۴-۵-۳: OpenMP

این یک استاندارد بسیار محبوب برای برنامه‌نویسی علمی و مهندسی در زبان‌های **C, C++, و FORTRAN** است.

ایده: به جای نوشتن کد پیچیده نخ‌سازی، فقط با یک "دستور" (Directive) به کامپایلر می‌گوییم "این بخش از کد را موازی کن".

- **`#pragma omp parallel`:** کامپایلر وقتی این دستور را می‌بیند، به طور خودکار یک تیم از نخ‌ها (معمولاً به تعداد هسته‌های CPU) ایجاد می‌کند و *همان* بلاک کد را توسط *همه* نخ‌ها به صورت موازی اجرا می‌کند.
- **`#pragma omp parallel for`:** این دستور جادویی است. اگر بالای یک حلقه `for` گذاشته شود، کامپایلر *خودش* تکرارهای حلقه را بین نخ‌ها تقسیم می‌کند.

مثال: جمع دو آرایه:

```
// این حلقه به صورت خودکار بین ۴ هسته تقسیم می‌شود
#pragma omp parallel for
```

```
} (++for (i=0; i < N; i
;[c[i] = a[i] + b[i
{
```

- **مزیت:** سادگی فوق‌العاده. شما فقط "چه" کاری باید موازی شود را مشخص می‌کنید، "چگونگی" آن را OpenMP انجام می‌دهد.

۴-۵-۴: اعزام مرکزی بزرگ (Grand Central Dispatch - GCD)

این تکنولوژی اپل برای **macOS** و **iOS** است و یکی از بهترین پیاده‌سازی‌های نخ‌سازی ضمنی محسوب می‌شود.

ایده: شبیه انبار نخ است، اما بسیار پیشرفته‌تر. شما وظایف خود را (که "بلاک" یا "بستار" نامیده می‌شوند) به یک "صف اعزام" (Dispatch Queue) می‌سپارید.

GCD دو نوع صف دارد:

1. **صف سِری (Serial Queue):** (خصوصی)
   - وظایف را به ترتیب FIFO (نوبت) اجرا می‌کند.
   - در هر لحظه **فقط یک** وظیفه در حال اجراست.
   - عالی برای "همگام‌سازی" و جلوگیری از تداخل (که در فصل‌های بعد می‌خوانیم).
1. **صف همروند (Concurrent Queue):** (عمومی)
   - وظایف را به ترتیب FIFO از صف برمی‌دارد، اما آن‌ها را به صورت **موازی** اجرا می‌کند (چندین وظیفه همزمان).
   - این صف‌ها دارای "کیفیت سرویس" (QOS) هستند تا اولویت‌بندی کنید:
     - `USER_INTERFACE`: بالاترین اولویت (برای رابط کاربری که قفل نکند).
     - `USER_INITIATED`: کارهایی که کاربر شروع کرده (مثل باز کردن فایل).
     - `UTILITY`: کارهای طولانی‌مدت (مثل دانلود فایل).
     - `BACKGROUND`: کمترین اولویت (مثل ایندکس کردن فایل‌ها).

GCD خودش یک انبار نخ (از Pthreads) را مدیریت می‌کند و به صورت هوشمند تعداد نخ‌ها را بر اساس بار سیستم کم و زیاد می‌کند.

۴-۵-۵: بلاک‌های سازنده نخ اینتل (TBB)

این یک کتابخانه مدرن **C++** از شرکت اینتل است. (شبیه OpenMP اما به صورت کتابخانه، نه دستور کامپایلر).

ایده: TBB هم الگوهای موازی‌سازی آماده در اختیار شما می‌گذارد.

- **`parallel_for`:** محبوب‌ترین الگوی آن است.
- شما یک "بازه" (Range) (مثلاً از ۰ تا n) و یک "بدنه" (Body) (که معمولاً یک تابع لامبدا است) به آن می‌دهید.
- TBB خودش بازه را به تکه‌های کوچکتر (Tasks) تقسیم می‌کند و آن‌ها را در انبار نخ داخلی‌اش اجرا می‌کند.

- **مزیت:** بسیار هوشمند است و "آگاه به کَش" (Cache-aware) عمل می‌کند تا بهترین کارایی را از سخت‌افزار بیرون بکشد.